

Debug 记录

1. 按键全部失灵

问题：程序运行正常、方块自动下落正常，但所有键盘按键（方向键、空格、字母键）均无响应。

原因：输入处理基于 `pygame.event.get()` 的 `KEYDOWN` 事件，但中文 Windows 的微软拼音等输入法会拦截键盘事件，导致 `pygame` 收不到 `KEYDOWN`。

修复：改用 `pygame.key.get_pressed()` 每帧轮询物理键盘状态，完全绕开事件系统。

```
# 旧：事件驱动
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_LEFT: ...

# 新：轮询
def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]: ...
```

同时在 `__init__` 中加入：

```
pygame.key.stop_text_input() # 禁用文本输入，防止 IME 拦截
```

2. 按键间歇性失灵 / 连续按住不响应

问题：操作频率较高时按键仍然失灵，长按方向键无法连续移动（只能移一两格就停了）。

原因：DAS 三阶段状态机逻辑有 bug——首次延迟触发后 `_das_time` 被重置，下一次循环又回到阶段 1 重新等 170ms，永远进不了连续重复阶段。

```
# 旧逻辑（有 bug）
elif now - self._das_time[k] >= self._das_delay:
    if now - self._das_time[k] >= self._das_interval:
```

```
self.move(dx, dy)
self._das_time[k] = now # ← 重置后又要等 170ms，陷入死循环
```

修复：引入 `_das_fired` 标记位，将状态机拆为三个阶段：

```
# 新逻辑
if not self._key_prev[k]:
    # 阶段 1: 首次按下 → 立即移动，进入等待
    self.move(dx, dy)
    self._das_time[k] = now
    self._das_fired[k] = False
elif not self._das_fired[k]:
    # 阶段 2: 超过 170ms → 移动一次，进入连续阶段
    if now - self._das_time[k] >= self._das_delay:
        self.move(dx, dy)
        self._das_time[k] = now
        self._das_fired[k] = True # ← 关键：标记进入阶段 3
else:
    # 阶段 3: 连续重复 → 每 50ms 移动一次
    if now - self._das_time[k] >= self._das_interval:
        self.move(dx, dy)
        self._das_time[k] = now
```

3. ESC 键无法退出

问题：窗口右上角 × 能关闭，但按 ESC 键无任何反应。

原因：从事件驱动改轮询后，`_track_keys` 列表没有 `K_ESCAPE`，`handle_input()` 里也没有 ESC 的处理逻辑。旧代码里 ESC 在 `KEYDOWN` 事件里处理，那条逻辑在重构时被删掉了。

修复：三步到位。

```
# ① _track_keys 加入 K_ESCAPE
self._track_keys = [..., pygame.K_ESCAPE]

# ② handle_input() 最顶部添加 ESC 检测（任何时候都生效）
if keys[pygame.K_ESCAPE] and not self._key_prev[pygame.K_ESCAPE]:
    self._quit = True
    return

# ③ run() 主循环条件加入 self._quit 检查
while running and not self._quit:
```

4. AI 模式在约 3400 分时自动停止

问题：AI 跑一段时间后突然不动了，看起来像卡住了。

原因：`reset()` 方法里写死了 `self.ai_mode = False`。AI 游戏结束后会调用 `reset()` 自动重开，重置过程中 AI 模式被意外关闭，游戏切回手动模式。

```
# 旧代码
def reset(self):
    ...
    self.ai_mode = False # ← 把 AI 关了!
```

修复：重置前保存、重置后恢复。

```
def reset(self):
    was_ai = getattr(self, 'ai_mode', False) # 保存
    ...
    self.ai_mode = False # 正常初始化
    ...
    self.ai_mode = was_ai # 恢复
```

5. 代码缩进错误导致 SyntaxError

问题：修改大段注释后 `py_compile.compile()` 报错 `IndentationError: unexpected indent (line 788)`。

原因：用 Edit 工具替换大段代码时，旧代码的残留片段没有被完全覆盖，导致重复的 `return` 语句存在错误的缩进。

修复：删除残留的旧代码行。

6. 右侧面板文字显示不全

问题：界面上的操作提示只能看到一半，"↓加速"及以下行被截断。

原因：提示文字起始 `ty = 510`，`8 行 × 25px 间距 = 终点 710px`，远超窗口高度 `600px`。

修复：压缩右侧面板布局。标签改用小号字体、紧凑间距（20px）、上移各区块的起始 y 坐标。
最终终点约 585px，完整可见。